MÁSTER FP IA Y BIG DATA · MÓDULO 5073 · BLOQUE 4

Sesión 2 de 6

Salidas estructuradas y uso de herramientas

Del texto libre a datos validados y acciones

Requisitos: sesión 1 completada y tostadero.db creada (python crear_bd.py)

Cámara de Madrid · Curso 2026-2027 · Martes 18:00–21:30

18:00 · REPASO DE LA SESIÓN 1

Lo que ya sabéis y lo que falta



Hablar con el modelo

Tres APIs, mismos conceptos, tras una interfaz común. La API no guarda estado, se paga por tokens y no es determinista.



Vuestra comparativa

Coste, latencia y calidad de tres proveedores. ¿Qué modelo elegisteis para 2.000 correos al mes, y con qué datos?



Lo que falta

Hasta ahora el modelo devuelve texto libre. Hoy: datos que cumplen un esquema y peticiones para ejecutar funciones.

ARQUITECTURA OBJETIVO

Dónde estamos



OBJETIVOS DE HOY

Al terminar la sesión serás capaz de…

1. Definir esquemas con Pydantic y JSON Schema y obtener respuestas que los cumplan en los tres proveedores.
2. Distinguir la validación de formato de la validación de negocio, y medir la calidad con un conjunto de evaluación.
3. Explicar el ciclo de function calling y programarlo sin frameworks.
4. Diseñar herramientas que el modelo use bien: nombres, descripciones, parámetros y errores.
5. Identificar los riesgos de dar herramientas a un modelo y aplicar las primeras salvaguardas.

AGENDA

Tres horas, dos teorías y dos prácticas

Hora	Bloque	Formato
18:00	Repaso de la sesión 1 y conclusiones de la comparativa	Debate
18:10	Teoría 1–2: salidas estructuradas	Teoría
18:35	P1 clasificador de correos con evaluación	Práctica
19:35	Descanso	—
20:05	Teoría 3–6: function calling y diseño de herramientas	Teoría
20:25	P2 herramientas · P3 bucle con OpenAI · P4 con Claude o Gemini	Práctica
21:20	Preguntas de control	Debate

TEORÍA · 18:10 · APARTADOS 1 Y 2

Salidas estructuradas

Del texto libre a datos que cumplen un esquema

1 · DEL TEXTO LIBRE A LOS DATOS

Cada correo debe acabar en un registro

Correo de un cliente

«Mandadme 5 kg de Etiopía y, de paso, la cafetera gotea. Lumbre.»

Registro (ilustrativo)

```
{
  "tipo": "mixto",
  "cliente": "Lumbre",
  "lineas": [{"producto": "Etiopía",
              "cantidad": 5}],
  "urgente": false,
  "resumen": "Pide 5 kg de Etiopía
    y avisa de que la cafetera gotea."
}
```

Tipo de petición, cliente, productos y urgencia. Con expresiones regulares es **frágil**; con un LLM hay **tres niveles de garantía**.

Tres niveles de garantía

1

Pedir JSON en el prompt

Suele funcionar, pero no hay garantía de que el JSON sea válido ni de que tenga los campos esperados.

Garantiza: nada

2

Modo JSON

El proveedor garantiza JSON válido, pero no su estructura.

Garantiza: JSON válido

3

Salidas estructuradas

La generación se restringe al esquema que envías: la respuesta siempre tiene la forma pedida.

Garantiza: la forma del esquema

Una clase Pydantic, dos usos

```
class Linea(BaseModel):
    producto: str
    cantidad: int

class CorreoClasificado(BaseModel):
    tipo: Literal["pedido", "incidencia",
                  "consulta", "mixto"]
    cliente: str | None = Field(
        description="Nombre del negocio
            si aparece; si no, null")
    lineas: list[Linea] = Field(
        description="Vacía si no hay pedido")
    urgente: bool
    resumen: str = Field(
        description="Una sola frase en español")
```

Genera el JSON Schema que recibe el modelo: model_json_schema().

Valida lo que devuelve el modelo y te entrega un objeto Python.

Literal cierra las opciones; **str | None** permite responder null.

JSON Schema reaparecerá en las herramientas y en MCP.

1 · DEL TEXTO LIBRE A LOS DATOS

El esquema garantiza la forma, no la verdad

El modelo puede rellenar un campo con un valor verosímil pero falso. Tres defensas:

Campos con null

«null si no aparece»: el modelo no tiene que inventarse el dato.

Enumeraciones

Cierran las opciones: pedido, incidencia, consulta o mixto.

Validación de negocio

Después del esquema: cantidades positivas, cliente que exista en la base de datos.

Y siempre, un conjunto de casos para medir la tasa de acierto.

2 · SALIDAS ESTRUCTURADAS EN LOS TRES SDK

Mismo esquema, tres llamadas

Proveedor	Llamada	Resultado
OpenAI	client.responses.parse(..., text_format=Modelo)	r.output_parsed (instancia de Pydantic)
Claude	client.messages.parse(..., output_format=Modelo); en la API, output_config.format	r.parsed_output
Gemini	client.interactions.create(..., response_format={"type": "text", "mime_type": "application/json", "schema": ...})	Modelo.model_validate_json(r.output_text)
Chat Completions (Gemini en clase)	client.chat.completions.parse(..., response_format=Modelo)	r.choices[0].message.parsed

Cada proveedor admite un **subconjunto de JSON Schema** (no todos aplican restricciones numéricas o formatos). Mantén los esquemas sencillos y valida el resto con Pydantic después.

PRÁCTICA · 18:35 · 60 MIN

P1 · Clasificador de correos con evaluación

Punto de partida

```
python s2_extraer_correo.py
```

CASOS trae 6 correos con su tipo esperado. El script los pasa por los tres proveedores e imprime los aciertos de cada uno.

OpenAI / Claude / Gemini = formato de API + modelo. En clase van por OpenRouter: gpt-5.4-mini, claude-haiku-4.5 y gemini-3.8-flash.

Tareas

1. Ejecuta el script y anota los aciertos de cada proveedor.
2. Amplía CASOS hasta al menos 15, con casos difíciles: ironía, dos clientes en el mismo correo, un correo en inglés, un correo sin nombre de negocio.
3. Revisa cada fallo: ¿error del modelo o del caso (etiqueta ambigua)? Corrige las etiquetas dudosas antes de comparar.
4. Añade una validación de negocio: si hay cliente, comprueba con buscar_cliente que existe en la base de datos.

P1 · CONFIGURACIÓN EN CLASE

Tres modelos, una clave de OpenRouter

Proveedor	SDK y API	Modelo en .env
OpenAI	openai · Responses API	OPENAI_MODEL=openai/gpt-5.4-mini
Claude	anthropic · Messages API	ANTHROPIC_MODEL=anthropic/claude-haiku-4.5
Gemini	openai · Chat Completions	GEMINI_MODEL=google/gemini-3.8-flash

Depende del modelo

Salidas estructuradas y herramientas dependen del modelo elegido. Si output_parsed, parsed_output o message.parsed llega como None, la evaluación falla en resultado.tipo: revisa el modelo de .env.

Gemini, por Chat Completions

En clase: chat.completions.parse(response_format=CorreoClasificado) con el SDK de OpenAI. La API nativa (Interactions) solo si rellenas la GEMINI_API_KEY opcional.

P1 · S2_EXTRAER_CORREO.PY (SIMPLIFICADO)

La misma extracción, tres llamadas

OpenAI

```
c = openai.OpenAI()
r = c.responses.parse(
  model=modelo,
  instructions=SISTEMA,
  input=correo,
  text_format=
    CorreoClasificado)

return r.output_parsed
```

Claude

```
c = anthropic.Anthropic()
r = c.messages.parse(
  model=modelo,
  max_tokens=1024,
  system=SISTEMA,
  messages=[{"role": "user",
    "content": correo}],
  output_format=
    CorreoClasificado)

return r.parsed_output
```

Gemini (Chat Completions)

```
c = openai.OpenAI(
  base_url=OPENROUTER,
  api_key=clave)
r = c.chat.completions.parse(
  model=modelo,
  messages=[
    {"role": "system", ...},
    {"role": "user", ...}],
  response_format=
    CorreoClasificado)

return (r.choices[0]
  .message.parsed)
```

P1 · EVALUACIÓN (SIMPLIFICADO)

Medir aciertos, no impresiones

```
for nombre, extraer in PROVEEDORES:
    aciertos = 0
    for correo, esperado in CASOS:
        resultado = extraer(correo)
        aciertos += resultado.tipo == esperado
        # el esquema garantiza la forma,
        # no que tenga sentido
        if any(l.cantidad <= 0
               for l in resultado.lineas):
            print("cantidad no válida")
    print(f"{nombre}: {aciertos}/{len(CASOS)}")
```

CASOS = pares (correo, tipo esperado). Seis de partida; ampliad a 15–20.

Casos difíciles: ironía, dos clientes, un correo en inglés, sin nombre de negocio.

Validación de negocio además del esquema: cantidades positivas y cliente que existe.

s2_extraer_correo.py


```
"""Sesión 2. Salidas estructuradas: de un correo libre a un objeto validado."""
import os
from typing import Literal

import anthropic
import openai
from dotenv import load_dotenv
from google import genai
from pydantic import BaseModel, Field

load_dotenv()


class Linea(BaseModel):
    producto: str = Field(description="Producto tal y como lo menciona el cliente")
    cantidad: int = Field(description="Unidades o kilos pedidos")
```

s2_extraer_correo.py


```
class CorreoClasificado(BaseModel):
    tipo: Literal["pedido", "incidencia", "consulta", "mixto"]
    cliente: str | None = Field(description="Nombre del negocio si aparece; si no, null")
    lineas: list[Linea] = Field(description="Vacía si no hay pedido")
    urgente: bool
    resumen: str = Field(description="Una sola frase en español")


SISTEMA = "Extraes información de correos de clientes de un distribuidor de café para hostelería."


def con_openai(correo: str) -> CorreoClasificado:
    r = openai.OpenAI().responses.parse(
        model=os.environ["OPENAI_MODEL"], instructions=SISTEMA, input=correo,
        text_format=CorreoClasificado,
    )
    return r.output_parsed
```

s2_extraer_correo.py


```
def con_claude(correo: str) -> CorreoClasificado:
    r = anthropic.Anthropic().messages.parse(
        model=os.environ["ANTHROPIC_MODEL"], max_tokens=1024, system=SISTEMA,
        messages=[{"role": "user", "content": correo}],
        output_format=CorreoClasificado,
    )
    return r.parsed_output


def con_gemini(correo: str) -> CorreoClasificado:
    if os.getenv("GEMINI_API_KEY"): # opcional: API nativa de Gemini
        cliente = genai.Client() # en una variable: si no, el SDK cierra el cliente antes de usarlo
        r = cliente.interactions.create(
            model=os.environ["GEMINI_MODEL"].removeprefix("google/"),
            system_instruction=SISTEMA, input=correo,
            response_format={
                "type": "text",
```

s2_extraer_correo.py


```
                "mime_type": "application/json",
                "schema": CorreoClasificado.model_json_schema(),
            },
        )
        return CorreoClasificado.model_validate_json(r.output_text)
    # Por defecto, vía OpenRouter con Chat Completions
    cliente = openai.OpenAI(base_url="https://openrouter.ai/api/v1",
                            api_key=os.environ["OPENROUTER_API_KEY"])
    r = cliente.chat.completions.parse(
        model=os.environ["GEMINI_MODEL"],
        messages=[{"role": "system", "content": SISTEMA}, {"role": "user", "content": correo}],
        response_format=CorreoClasificado,
    )
    return r.choices[0].message.parsed


# Mini conjunto de evaluación: (correo, tipo esperado). Amplíalo a 15-20 casos.
CASOS = [
```

s2_extraer_correo.py


```
    ("Necesito 20 kg del blend de hostelería para el viernes. Bar La Esquina.", "pedido"),
    ("El molino hace un ruido raro desde ayer, ¿podéis venir?", "incidencia"),
    ("¿Tenéis café descafeinado de especialidad?", "consulta"),
    ("Mandadme 5 kg de Etiopía y, de paso, la cafetera gotea. Lumbre.", "mixto"),
    ("URGENTE: sin café para mañana, 10 kg de blend. Hotel Prado Suites", "pedido"),
    ("Buenas, ¿la factura de septiembre ya está emitida?", "consulta"),
]

if __name__ == "__main__":
    for nombre, extraer in [("OpenAI", con_openai), ("Claude", con_claude), ("Gemini", con_gemini)]:
        aciertos = 0
        for correo, esperado in CASOS:
            resultado = extraer(correo)
            aciertos += resultado.tipo == esperado
            # Validación de negocio: el esquema garantiza la forma, no que tenga sentido
            if any(l.cantidad <= 0 for l in resultado.lineas):
                print(f" [{nombre}] cantidad no válida en: {correo[:40]}...")
        print(f"{nombre}: {aciertos}/{len(CASOS)} tipos correctos")
```

19:35 · 30 MINUTOS

Descanso

A las 20:05, teoría: function calling y diseño de herramientas.

TEORÍA · 20:05 · APARTADOS 3 A 6

Function calling y diseño de herramientas

El modelo pide, tu código ejecuta

3 · FUNCTION CALLING

El modelo pide, tu código ejecuta

APLICACIÓN

1

Envía la pregunta y la lista de herramientas: nombre, descripción y parámetros en JSON Schema.

MODELO

2

Decide si necesita alguna. Si es así, en lugar de texto devuelve peticiones de llamada con argumentos.

APLICACIÓN

3

Valida los argumentos y ejecuta la función real: una consulta SQL, una API, un cálculo.

APLICACIÓN

4

Devuelve el resultado al modelo, asociado al identificador de la llamada.

MODELO

5

Continúa: pide otra herramienta o redacta la respuesta final.

El modelo nunca ejecuta nada. Todo lo que ocurre en el mundo real lo hace tu código, y ahí residen el control y la responsabilidad.

3 · FUNCTION CALLING

Dos llamadas encadenadas

«¿Qué pedidos pendientes tiene el Bar La Esquina y cuánto suman?»

Modelo pide	`buscar_cliente(texto="La Esquina")`
Tu código	`[{"id": 1, "nombre": "Bar La Esquina", "tipo": "bar", "distrito": "Retiro"}]`
Modelo pide	`consultar_pedidos(cliente_id=1)`
Tu código	`[{"id": 1001, "estado": "entregado", ...},` ` {"id": 1004, "estado": "pendiente", "total": 98.0}]`
Modelo responde	Texto final: un pedido pendiente, el 1004, por 98 €.

4 · EL MISMO CONCEPTO EN TRES DIALECTOS

Mismos pasos, otros nombres

	OpenAI (Responses)	Claude (Messages)	Gemini (Interactions)
Definir la herramienta	{"type": "function", "name", "description", "parameters", "strict"}	{"name", "description", "input_schema"}	{"type": "function", "name", "description", "parameters"}
Señal de llamada	elementos de r.output con type == "function_call"	r.stop_reason == "tool_use"	pasos de r.steps con type == "function_call"
Datos de la llamada	name, call_id, arguments (cadena JSON)	bloque tool_use: name, id, input (diccionario)	name, id, arguments (diccionario)
Devolver el resultado	{"type": "function_call_output", "call_id", "output"}	mensaje de usuario con {"type": "tool_result", "tool_use_id", "content"}	{"type": "function_result", "name", "call_id", "result"} con previous_interaction_id

Cuarto dialecto, Chat Completions (Gemini en clase vía OpenRouter, Ollama): para_chat() envuelve la definición en {"type": "function", "function": {...}}; las llamadas llegan en message.tool_calls, con arguments como cadena JSON, y el resultado vuelve como {"role": "tool", "tool_call_id", "content"}.

La definición la lee el modelo

```
{"name": "crear_incidencia",
 "description": "Registra una avería de una
    máquina de un cliente. Úsala solo si el
    cliente describe una avería concreta.",
 "parameters": _obj({
    "cliente_id": {"type": "integer"},
    "maquina": {"type": "string",
        "description": "SKU de la máquina,
            p. ej. MAQ-010"},
    "descripcion": {"type": "string"},
    "prioridad": {"type": "string",
        "enum": ["baja", "media", "alta"]},
 })}
```

Nombre con verbo y objeto: buscar_cliente, crear_incidencia.

Descripción: qué hace, cuándo usarla y cuándo no, qué devuelve y en qué orden llamarla.

Parámetros tipados y cerrados: enumeraciones, identificadores enteros y formato descrito.

5 · DISEÑAR HERRAMIENTAS QUE EL MODELO USE BIEN

Pocas, compactas y con errores útiles

Pocas y bien separadas

Cada definición ocupa tokens en todas las llamadas y aumenta la probabilidad de elegir mal.

Errores útiles

«No existe el cliente 99. Búscalo antes con buscar_cliente» permite que el modelo se corrija solo.

Resultados compactos

Devuelve solo los campos necesarios.

Control de la elección

Automática (por defecto), obligatoria o desactivada; llamadas en paralelo cuando son independientes.

Herramientas integradas del proveedor (búsqueda web, ejecución de código, búsqueda en ficheros): las ejecuta el proveedor, no tu código. Solo con claves nativas: vía OpenRouter no están disponibles con estos SDK y no se usan en el curso.

6 · SEGURIDAD DESDE EL PRIMER DÍA

Cuatro salvaguardas desde hoy

Valida los argumentos

Como si vinieran de un usuario anónimo: el modelo puede inventar identificadores o fechas.

Consultas parametrizadas

Nunca dejes que el modelo escriba SQL libre contra la base de datos de producción.

Mínimo privilegio

Una herramienta de consulta no necesita permisos de escritura.

Confirmación humana

Para las herramientas con efectos: crear, enviar, borrar. Se implementa en la sesión 3.

En herramientas.py, todas las consultas usan parámetros (?) y crear_incidencia comprueba que el cliente existe antes de insertar.

PRÁCTICA · 20:25 · 10 MIN

P2 · Herramientas como funciones Python

Cuatro funciones de negocio

`buscar_cliente(texto)`
clientes por nombre, coincidencia parcial

`consultar_pedidos(cliente_id)`
fecha, estado y total de cada pedido

`stock_producto(texto)`
precio y stock por nombre o SKU

`crear_incidencia(...)`
registra una avería: la única que escribe

Una definición, tres dialectos

```
def para_openai():
    return [{"type": "function", "strict": True,
             **d} for d in DEFINICIONES]
def para_claude():
    return [{"name": d["name"],
             "description": d["description"],
             "input_schema": d["parameters"]}
            for d in DEFINICIONES]
def para_gemini():
    return [{"type": "function", **d}
            for d in DEFINICIONES]
```

herramientas.py


```
"""Sesiones 2 y 3. Herramientas de negocio como funciones Python + su esquema JSON.

En la sesión 5 estas mismas funciones pasarán a un servidor MCP."""
import sqlite3
from datetime import date
from pathlib import Path

BD = Path(__file__).with_name("tostadero.db")


def _consulta(sql: str, params: tuple = ()) -> list[dict]:
    with sqlite3.connect(BD) as con:
        con.row_factory = sqlite3.Row
        return [dict(f) for f in con.execute(sql, params)]


def buscar_cliente(texto: str) -> list[dict]:
    return _consulta("SELECT id, nombre, tipo, distrito FROM clientes WHERE nombre LIKE ?",
```

herramientas.py


```
                    (f"%{texto}%",))


def consultar_pedidos(cliente_id: int) -> list[dict]:
    return _consulta("SELECT id, fecha, estado, total FROM pedidos WHERE cliente_id = ?",
                     (cliente_id,))


def stock_producto(texto: str) -> list[dict]:
    return _consulta("SELECT sku, nombre, precio, stock FROM productos "
                     "WHERE nombre LIKE ? OR sku LIKE ?", (f"%{texto}%", f"%{texto}%"))


def crear_incidencia(cliente_id: int, maquina: str, descripcion: str, prioridad: str) -> dict:
    if not _consulta("SELECT 1 FROM clientes WHERE id = ?", (cliente_id,)):
        return {"error": f"No existe el cliente {cliente_id}. Búscalo antes con buscar_cliente."}
    with sqlite3.connect(BD) as con:
        cur = con.execute(
```

herramientas.py


```
            "INSERT INTO incidencias (cliente_id, maquina, descripcion, prioridad, fecha) "
            "VALUES (?,?,?,?,?)",
            (cliente_id, maquina, descripcion, prioridad, date.today().isoformat()))
    return {"id": cur.lastrowid, "estado": "abierta"}


def _obj(props: dict) -> dict:
    """Esquema de objeto compatible con el modo estricto de OpenAI:
    todas las propiedades obligatorias y sin propiedades adicionales."""
    return {"type": "object", "properties": props,
            "required": list(props), "additionalProperties": False}


# Definición neutra: nombre, descripción (¡la lee el modelo!) y parámetros en JSON Schema
DEFINICIONES = [
    {"name": "buscar_cliente",
     "description": "Busca clientes por nombre (coincidencia parcial). Devuelve id, nombre, "
                    "tipo y distrito. Úsala antes de cualquier otra herramienta que pida cliente_id.",
```

herramientas.py


```
     "parameters": _obj({"texto": {"type": "string", "description": "Parte del nombre"}})},
    {"name": "consultar_pedidos",
     "description": "Lista los pedidos de un cliente con fecha, estado y total en euros.",
     "parameters": _obj({"cliente_id": {"type": "integer"}})},
    {"name": "stock_producto",
     "description": "Consulta precio y stock de productos cuyo nombre o SKU contenga el texto.",
     "parameters": _obj({"texto": {"type": "string"}})},
    {"name": "crear_incidencia",
     "description": "Registra una avería de una máquina de un cliente. Úsala solo si el cliente "
                    "describe una avería concreta.",
     "parameters": _obj({
         "cliente_id": {"type": "integer"},
         "maquina": {"type": "string", "description": "SKU de la máquina, p. ej. MAQ-010"},
         "descripcion": {"type": "string"},
         "prioridad": {"type": "string", "enum": ["baja", "media", "alta"]},
     })},
]
```

herramientas.py


```
FUNCIONES = {"buscar_cliente": buscar_cliente, "consultar_pedidos": consultar_pedidos,
             "stock_producto": stock_producto, "crear_incidencia": crear_incidencia}


# Adaptadores: misma herramienta, cuatro "dialectos"
def para_openai() -> list[dict]:
    return [{"type": "function", "strict": True, **d} for d in DEFINICIONES]


def para_claude() -> list[dict]:
    return [{"name": d["name"], "description": d["description"], "input_schema": d["parameters"]}
            for d in DEFINICIONES]


def para_gemini() -> list[dict]:
    return [{"type": "function", **d} for d in DEFINICIONES]
```

herramientas.py


```
def para_chat() -> list[dict]: # Chat Completions: Gemini vía OpenRouter, Ollama...
    return [{"type": "function", "function": d} for d in DEFINICIONES]
```

PRÁCTICA · 20:35 · 25 MIN

P3 · El bucle de herramientas con OpenAI

Ejecutar

```
python s2_chat_herramientas.py

# cada llamada deja una traza:
  -> nombre(args) = resultado
```

Cambia la pregunta en preguntar(...) y observa qué herramientas se llaman y en qué orden.

Prueba estas preguntas

1. «¿Qué pedidos pendientes tiene el Bar La Esquina y cuánto suman?» Requiere dos llamadas encadenadas.
2. «¿Cuánto stock queda de descafeinado?»
3. «¿Qué pedidos tiene el Bar Inexistente?» ¿Cómo gestiona el resultado vacío?
4. «Borra el pedido 1003». No existe esa herramienta: ¿qué responde?

P3 · S2_CHAT_HERRAMIENTAS.PY (SIMPLIFICADO)

El bucle, en quince líneas

```
def preguntar(pregunta, max_vueltas=5):
    entrada = [{"role": "user", "content": pregunta}]
    for _ in range(max_vueltas):
        r = cliente.responses.create(model=modelo, instructions=SISTEMA,
                                     tools=para_openai(), input=entrada)
        entrada += r.output  # devolvemos al modelo lo que acaba de producir
        llamadas = [i for i in r.output if i.type == "function_call"]
        if not llamadas: return r.output_text  # sin llamadas: respuesta final
        for llamada in llamadas:  # ¡quien ejecuta es nuestro código!
            args = json.loads(llamada.arguments)
            resultado = FUNCIONES[llamada.name](**args)
            entrada.append({"type": "function_call_output",
                            "call_id": llamada.call_id,
                            "output": json.dumps(resultado, ensure_ascii=False)})
    return "No he podido completar la consulta en el número de pasos permitido."
```

s2_chat_herramientas.py


```
"""Sesión 2. Function calling "a mano" con la Responses API de OpenAI."""
import json
import os

import openai
from dotenv import load_dotenv

from herramientas import FUNCIONES, para_openai

load_dotenv()
cliente = openai.OpenAI()
SISTEMA = ("Eres el asistente interno de Tostadero Retiro. Usa las herramientas para "
           "consultar datos reales; si no encuentras algo, dilo. No inventes cifras.")


def preguntar(pregunta: str, max_vueltas: int = 5) -> str:
    entrada: list = [{"role": "user", "content": pregunta}]
    for _ in range(max_vueltas):
```

s2_chat_herramientas.py


```
        r = cliente.responses.create(
            model=os.environ["OPENAI_MODEL"], instructions=SISTEMA,
            tools=para_openai(), input=entrada,
        )
        entrada += r.output # devolvemos al modelo lo que acaba de producir
        llamadas = [item for item in r.output if item.type == "function_call"]
        if not llamadas: # sin llamadas pendientes: es la respuesta final
            return r.output_text
        for llamada in llamadas: # ¡quien ejecuta es nuestro código, no el modelo!
            args = json.loads(llamada.arguments)
            resultado = FUNCIONES[llamada.name](**args)
            print(f" -> {llamada.name}({args}) = {str(resultado)[:80]}")
            entrada.append({
                "type": "function_call_output",
                "call_id": llamada.call_id,
                "output": json.dumps(resultado, ensure_ascii=False),
            })
    return "No he podido completar la consulta en el número de pasos permitido."
```

s2_chat_herramientas.py


```
if __name__ == "__main__":
    print(preguntar("¿Qué pedidos pendientes tiene el Bar La Esquina y cuánto suman?"))
    print(preguntar("¿Nos queda descafeinado? ¿A qué precio?"))
```

PRÁCTICA · 21:00 · 20 MIN

P4 · El mismo asistente con Claude o Gemini

Porta el bucle de P3 a Claude, con para_claude(), o a Gemini, con para_chat(), usando la tabla del apartado 4.

OpenAI / Claude / Gemini = formato de API + modelo. En clase van por OpenRouter: gpt-5.4-mini, claude-haiku-4.5 y gemini-3.8-flash.

Con Claude

- Señal: r.stop_reason == "tool_use"
- Bloques tool_use: name, id, input (diccionario)
- Resultado: mensaje de usuario con tool_result y tool_use_id

Con Gemini

- Chat Completions: señal en message.tool_calls
- function.name, id, function.arguments (cadena JSON)
- Resultado: mensaje con role "tool" y tool_call_id

Comparad con OpenAI

- Número de llamadas
- Tokens: imprime r.usage en cada vuelta
- Calidad de la respuesta final

21:20 · CIERRE

Preguntas de control

1. ¿Qué garantiza una salida estructurada y qué no garantiza?
2. En function calling, ¿quién ejecuta la función? ¿Qué consecuencias tiene para la seguridad?
3. ¿Por qué devolver «No existe el cliente 99. Búscalo antes con buscar_cliente» en lugar de lanzar una excepción?
4. Tu asistente tiene 40 herramientas. ¿Qué problemas prevés y qué harías?
5. Si el JSON siempre es válido, ¿para qué necesitas un conjunto de evaluación?

RECURSOS

Para seguir leyendo

Salidas estructuradas

Guías de OpenAI, Claude y Gemini: developers.openai.com/api/docs, platform.claude.com/docs y ai.google.dev/gemini-api/docs/structured-output

Function calling

Guías de uso de herramientas (tool use o function calling) de los tres proveedores, en las mismas documentaciones.

Pydantic y JSON Schema

docs.pydantic.dev y json-schema.org

OpenRouter

openrouter.ai/docs: apartados Structured Outputs y Tool Calling (qué modelos los admiten y cómo se piden).

ENTREGABLE E2 · 15 %

Clasificador de correos evaluado y asistente con herramientas

Carpeta e2/: s2_extraer_correo.py con al menos 15 casos, s2_chat_claude.py o s2_chat_gemini.py, y analisis.md con la evaluación (P1), las trazas (P3) y la comparativa con OpenAI (P4).

Repositorio de entregas · Domingo 25 de octubre, 23:59

Próxima sesión

Agentes I: el bucle agéntico, razonamiento y planificación

Requisito: sesiones 1 y 2 completadas. herramientas.py se reutiliza en la sesión 3.